03-Árboles Binarios

Diplomado Data Engineer

Universidad de Santiago de Chile

UNIVERSIDAD DE SANTIAGO DE CHILE

Juan Pablo Castillo

Problema de Búsqueda

Organizar y recuperar información es la esencia de la mayoría de las aplicaciones en informática.

Hoy en día, buscar datos en un conjunto de datos es una de las tareas más comúnmente realizadas.

Por ejemplo:

- Búsqueda en grandes bases de datos de personas
- Búsquedas en sistemas de transacciones
  - e.g., bancos
- Búsquedas en la web
- Búsquedas en redes sociales

Problema de Búsqueda

Estudiaremos estructuras especializadas que permiten resolver el problema de búsqueda de forma eficiente

Primero, definamos en que consiste el problema de búsqueda

- De forma simple, se puede definir como:

El proceso de determinar si un elemento en particular es miembro de un conjunto en particular.

Problema de Búsqueda

Definición Formal

Supongamos que tenemos una colección L de n datos de la siguiente forma:

$$L = \{(k_0, I_0), (k_1, I_1), \ldots, (k_{n-1}, I_{n-1})\}$$

- Donde I_j es la información o dato asociado a la clave k_j.
- Dada una clave en particular K, el **problema de búsqueda** consiste en localizar el registro (k_j, I_j) en L tal que $k_j = K$ (si es que existe).
- Estaremos interesados solo en búsquedas exactas.
  - i.e., K debe ser exactamente igual a k_j.

Problema de Búsqueda

<u>Técnicas para resolver el problema de búsqueda</u>

1. **Métodos basados en listas:**
   - Método simple, pero toma tiempo $O(n)$ en peor caso.
   - Toca recorrer toda la lista en el peor de los casos para determinar si la clave está o no.
2. **Métodos basados en árboles:**
   - Tiempo de búsqueda $O(\log_2 n)$.
3. **Métodos basados en acceso directo mediante la clave (hashing)**
   - Tiempo de búsqueda $O(1)$ en promedio.

Diccionarios

Diccionarios

Se define un Diccionario como una estructura de datos que almacena registros con la siguiente forma:

- `clave-valor`: (k, I), donde la clave k es única en cada registro.

Este tipo de estructuras soportan las operaciones:

1) Obtener el valor I asociado a la clave k.
2) Insertar un registro (k, I).
3) Borrar un regitro (k, I) mediante su clave k.

Diccionarios

Tipos de Diccionarios

Existen dos tipos de diccionarios:

1) **Diccionarios ordenados:** que tienen las claves ordenadas y permiten *iterar en orden* sobre estas de manera eficiente (en tiempo $O(n)$).

2) **Diccionarios desordenados:** que tienen las claves desordenadas. Sobre estos no se puede iterar en orden las claves eficientemente.

Árboles

Árboles

Las listas estudiadas anteriormente, son estructuras de datos lineales: existe exactamente un elemento previo y otro siguiente.

Pero las listas tienen una limitación importante (al menos en las que estudiamos, listas implementadas sobre arreglos):

- La inserción y borrado en posiciones arbitrarias no se pueden realizar de forma eficiente.
- Buscaremos un punto medio para que: insertar/borrar y además acceder a un elemento arbitrario sean “rápidos”.

Árboles

Árboles

- Las estructuras de tipo árbol nos permiten:
  - Acceso eficiente a los elementos.
  - Actualizar los valores de dichos elementos de forma eficiente.
- En este tipo de estructuras no existe la **secuencialidad/ linealidad** entre los elementos.
- En los árboles predominará una estructura jerárquica entre los elementos.
  - De esta forma un elemento tiene:
    - Un único antecesor, pero
    - Puede tener varios sucesores.

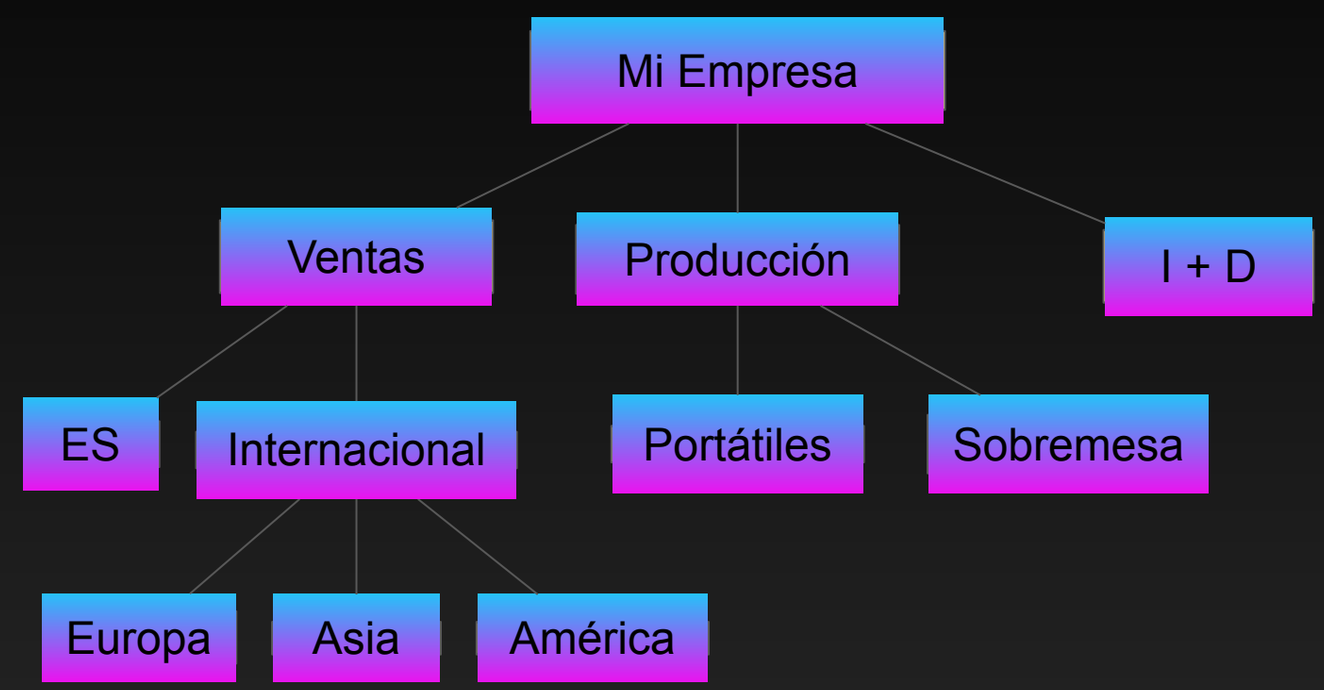
Árboles

Los árboles son muy utilizados en informática, algunos ejemplos:

- Árboles sintácticos, utilizados por compiladores para interpretar elementos sintácticos de un lenguaje, o representar expresiones aritméticas.
- Representar datos para permitir búsquedas y accesos a ellos relativamente rápido.
- Sistema de archivos:
    - Estructura de directorios y subdirectorios en forma de árbol para organizar la información.
- Diagramas de clases para programación orientada a objetos

Árboles: Terminología

- Un árbol está formado por conjunto finito de elementos, llamados nodos.
- Uno de esos elementos es identificado de manera especial como **raíz del árbol**.
- Existe una relación de padre-hijo que coloca a los nodos del árbol en una estructura jerárquica.



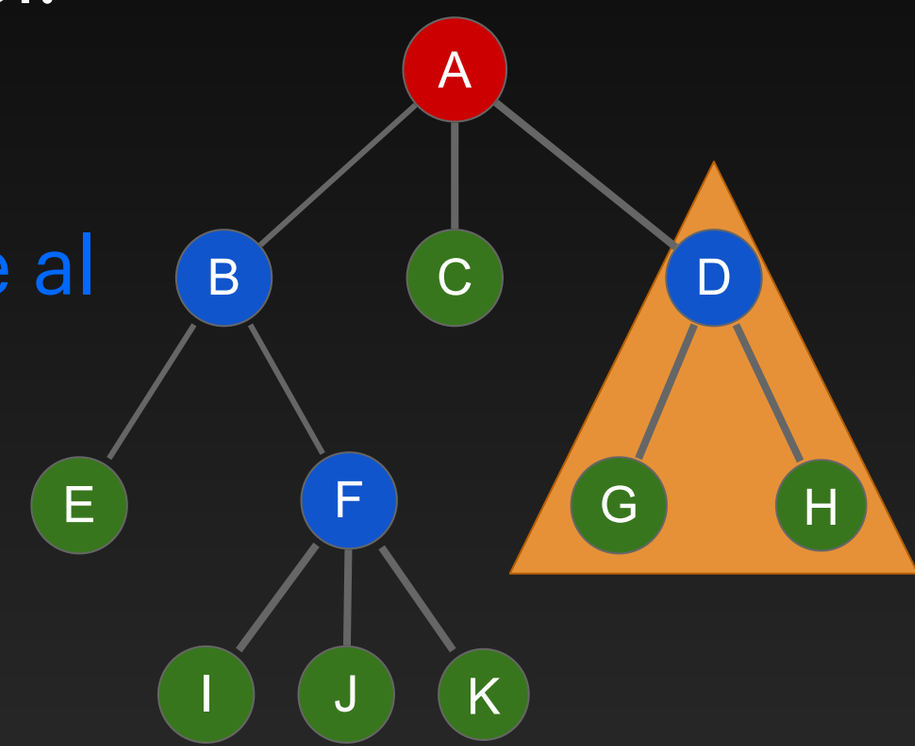
Note que la estructura resultante no es lineal:

- Cada nodo puede tener **varios sucesores**
- Cada nodo tiene un **único predecesor**

Árboles: Terminología

Los nodos se clasifican de acuerdo a la posición que ocupan dentro del árbol:

- **Raíz:** único nodo sin padre.
- **Nodo interno o nodo interior:** tiene al menos un hijo.
- **Descendiente directo:** hijo
- **Descendientes:** hijo, nieto, ...

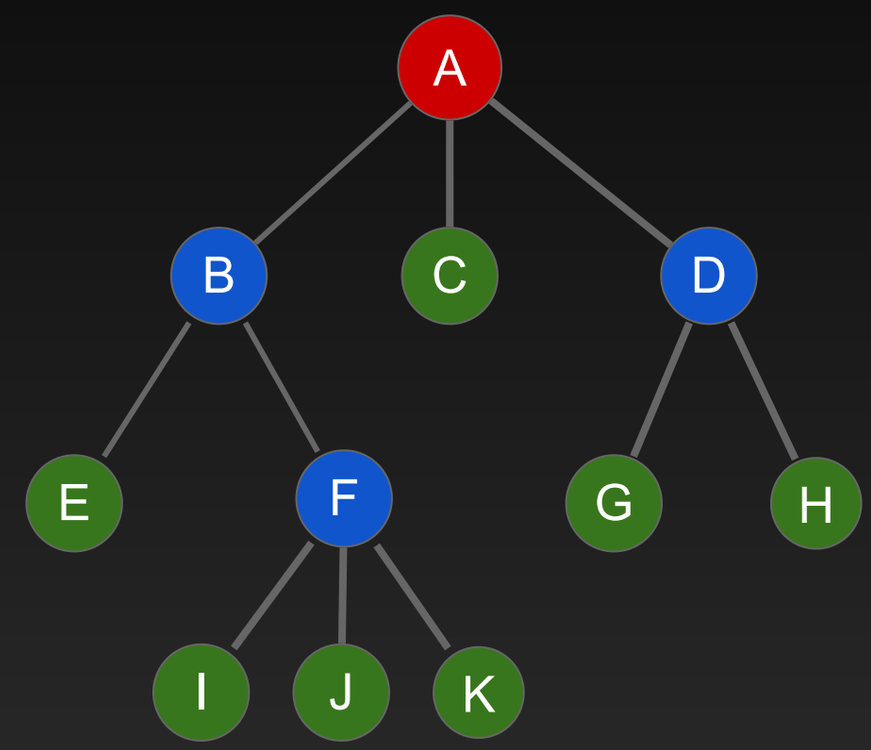


Si existe un camino desde el nodo u al nodo v en el árbol, entonces:

- v es un descendiente de u, y
- u es un ancestro de v

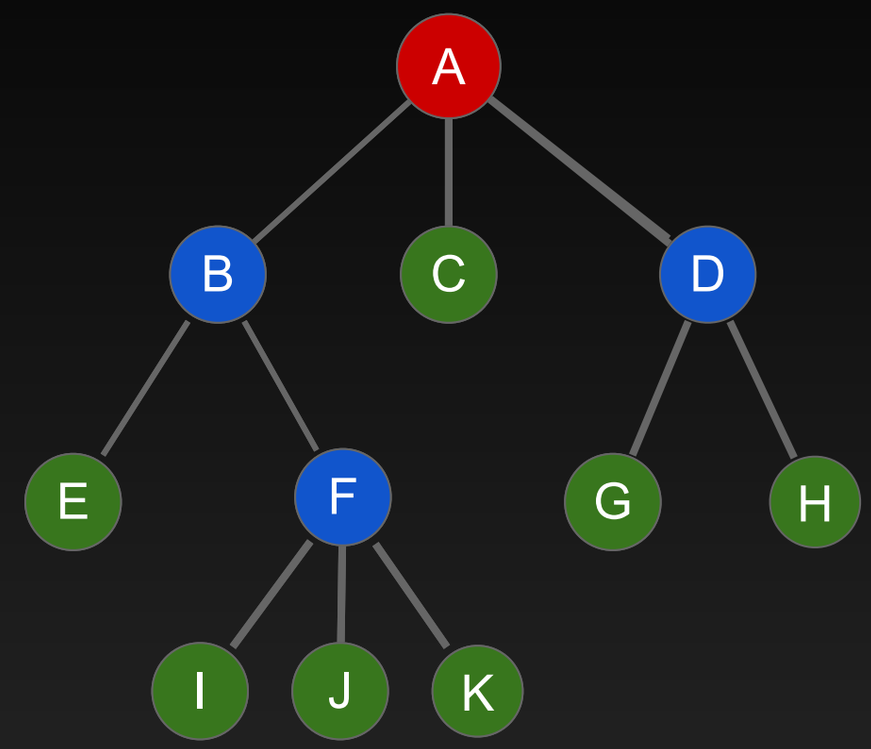
Árboles: Terminología

- **Camino** $\langle n_1, n_2, \ldots, n_k \rangle$**:** secuencia de nodos en un árbol tal que n_i es padre de n_{i+1} para $1 \leq i \leq k$.
  <u>Por ejemplo:</u>
  - $camino(A, K) = \{A, B, F, K\}$
  - $camino(D, K) = \{\}$
- **Largo del camino:** número de enlaces que hay desde el primer nodo hasta el último.
  - $\#nodos - 1$
- **Rama:** camino desde la **raíz** a una **hoja**.



Árboles: Terminología

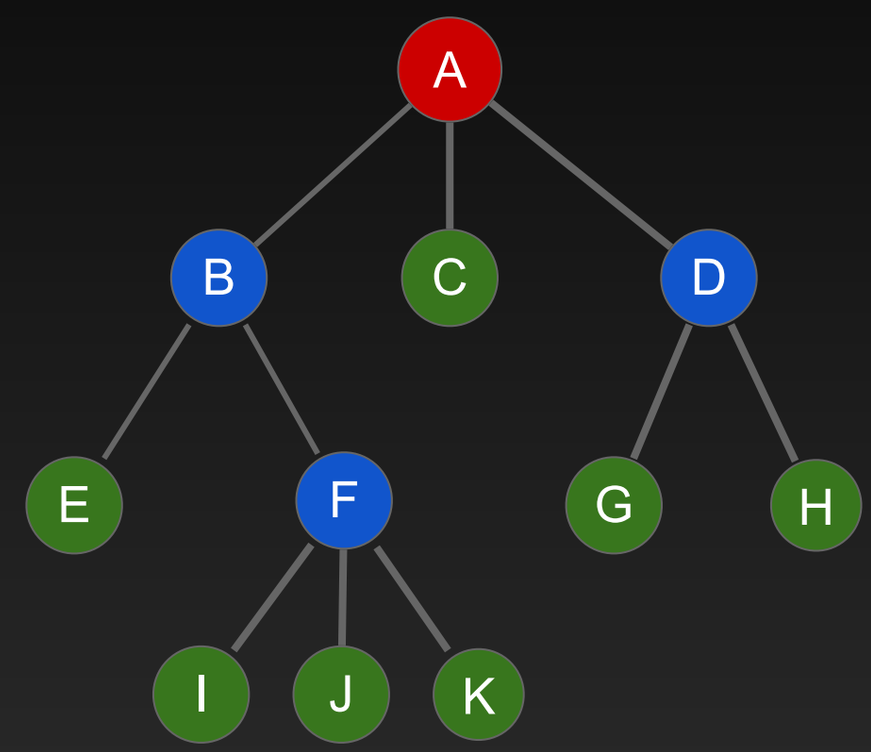
- **Grado de un nodo:** número de descendientes directos del nodo.
  - Número de subárboles o número de hijos del nodo.
  - Note que el grado de una hoja es 0.
- **Grado del árbol:** número máximo de hijos que tienen los nodos de un árbol
  - Una lista es un árbol de grado 1
  - **Árbol binario:** árbol de grado 2 (cada nodo tiene a lo más 2 hijos)
  - **Árbol multiario:** árbol de grado mayor a 2



- **Profundidad de un nodo:** número de predecesores.
  - Largo del único camino desde la raíz al nodo
  - <u>Ejemplos</u>:
    - $profundidad(A) = 0$
    - $profundidad(H) = 2$

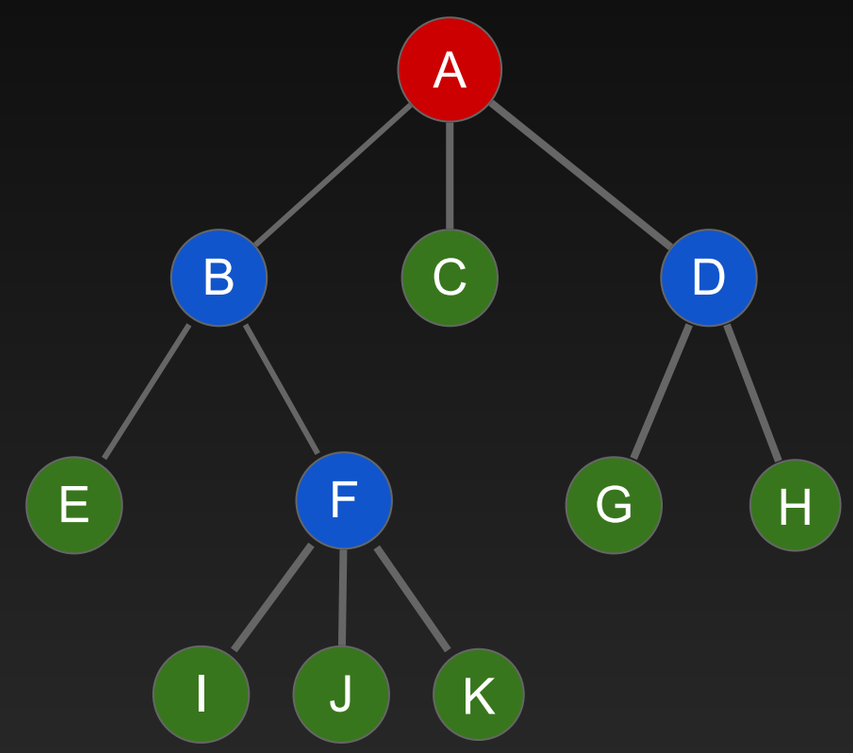
Árboles: Terminología

- **Altura de un árbol:**
  - Profundidad máxima de los nodos
  - Longitud del camino más largo (máximo número de enlaces) de las ramas del árbol.
- Por ejemplo, el árbol de la derecha tiene altura 3.
  - Las ramas $A - B - F - K$, $A - B - F - I$ y $A - B - F - J$ son las más largas



Árboles: Terminología

- **Nivel de un nodo:**
  - Número de nodos del camino desde la raíz hasta el nodo.
  - **Por convención, al nodo raíz se le considera el nivel cero del árbol.**
  - Los niveles del árbol se enumeran desde cero hasta la rama más larga del árbol



Por ejemplo:

`Nivel 0:` A

`Nivel 1:` B, C, D

`Nivel 2:` E, F, G, H

`Nivel 3:` I, J, K

Árboles: Recorridos

En algunas ocasiones se necesita procesar un árbol visitando sus nodos, y realizar alguna acción sobre ellos.

- Por ejemplo: mostrar su contenido, sumar valores guardados, etc.

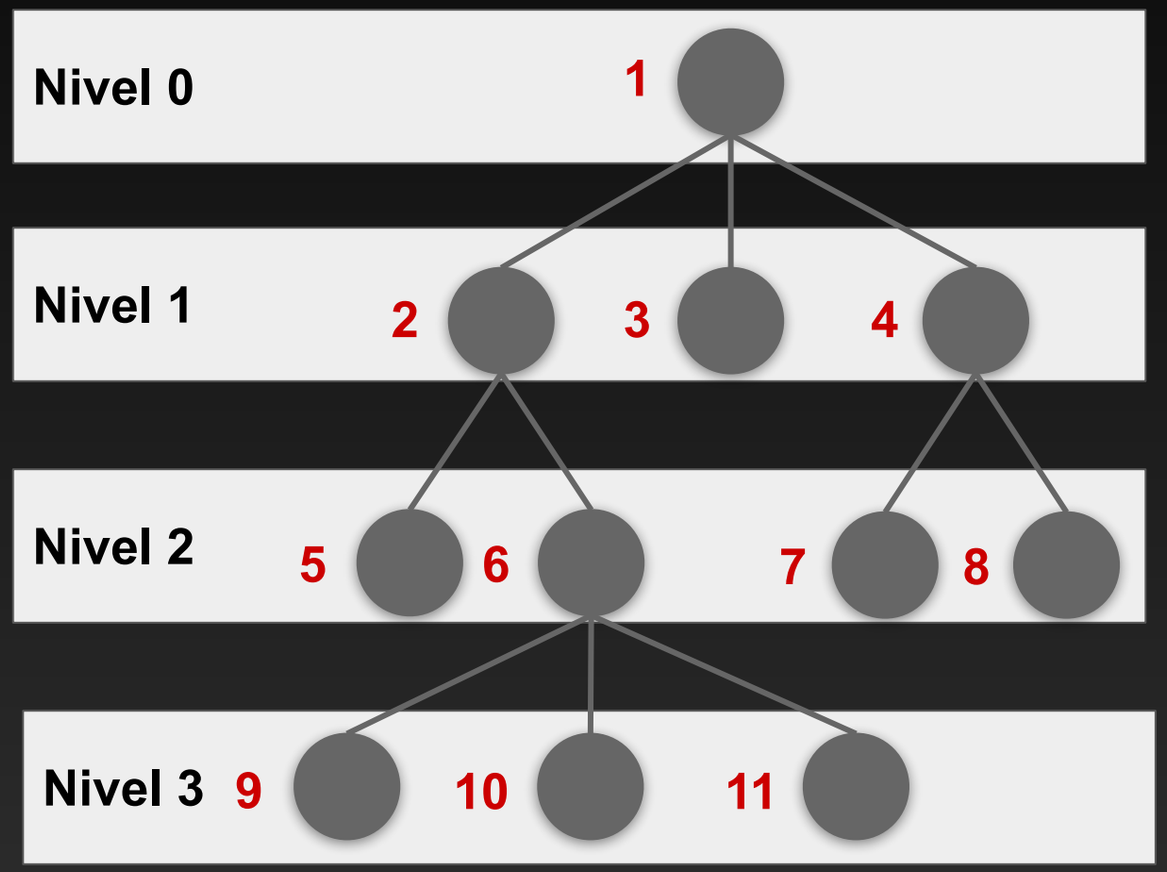
Existen dos maneras típicas de recorrer un árbol:

1. Recorrido en profundidad
2. Recorrido en anchura

Árboles: Recorrido en Anchura

El árbol se visita por niveles:

- Primero se visita la raíz, y se colocan todos sus hijos en una estructura tipo Cola.
- Por cada nodo en la cola, se procede de la misma manera.



Árboles: Recorrido en Profundidad

Consiste en moverse en cada paso del recorrido hacia nodos de mayor profundidad en el árbol, hasta que no se pueda avanzar en ese sentido (porque se alcanzó una hoja)

Se pueden llevar a cabo de tres maneras diferentes:

A. Recorrido **Pre-orden**

B. Recorrido **In-orden**

C. Recorrido **Post-orden**

Árboles: Recorrido en Profundidad

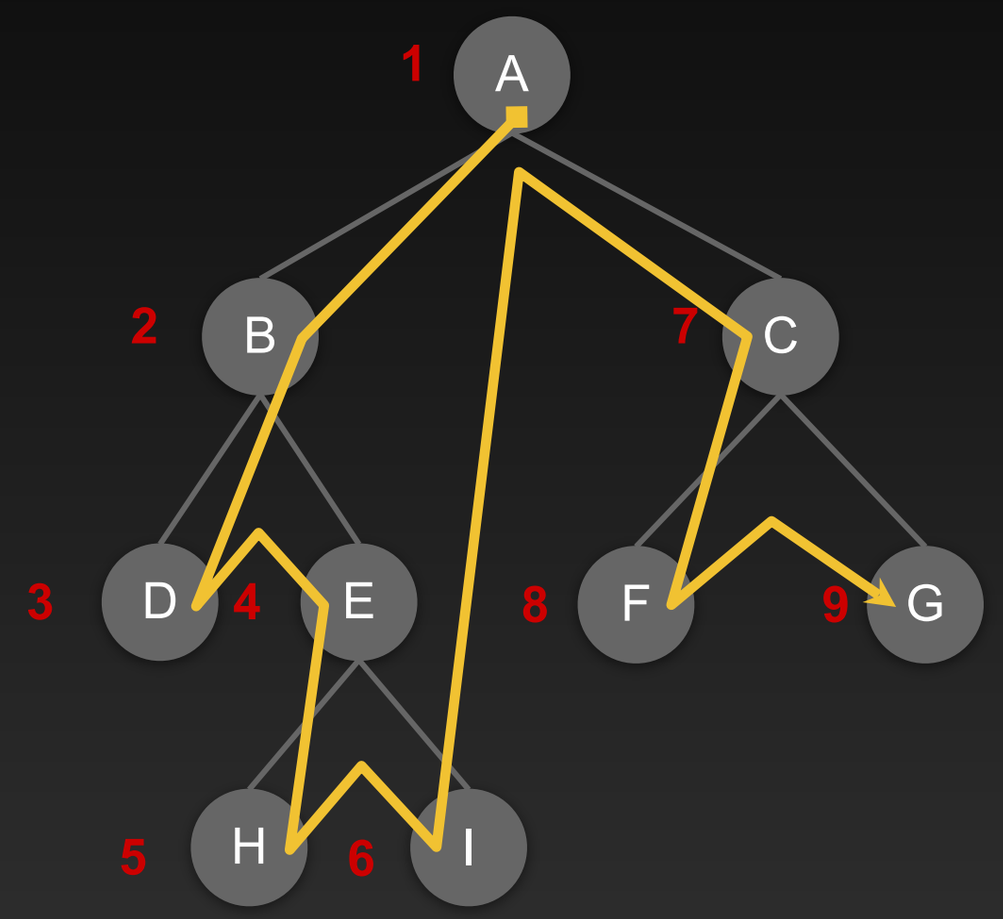
Recorrido en Pre-orden

Partiendo desde la raíz:

1) Visitar nodo actual

2) Visitar hijo izquierdo en pre-orden

3) Visitar hijo derecho en pre-orden



El recorrido en este caso sería: A-B-D-E-H-I-C-F-G

Árboles: Recorrido en Profundidad

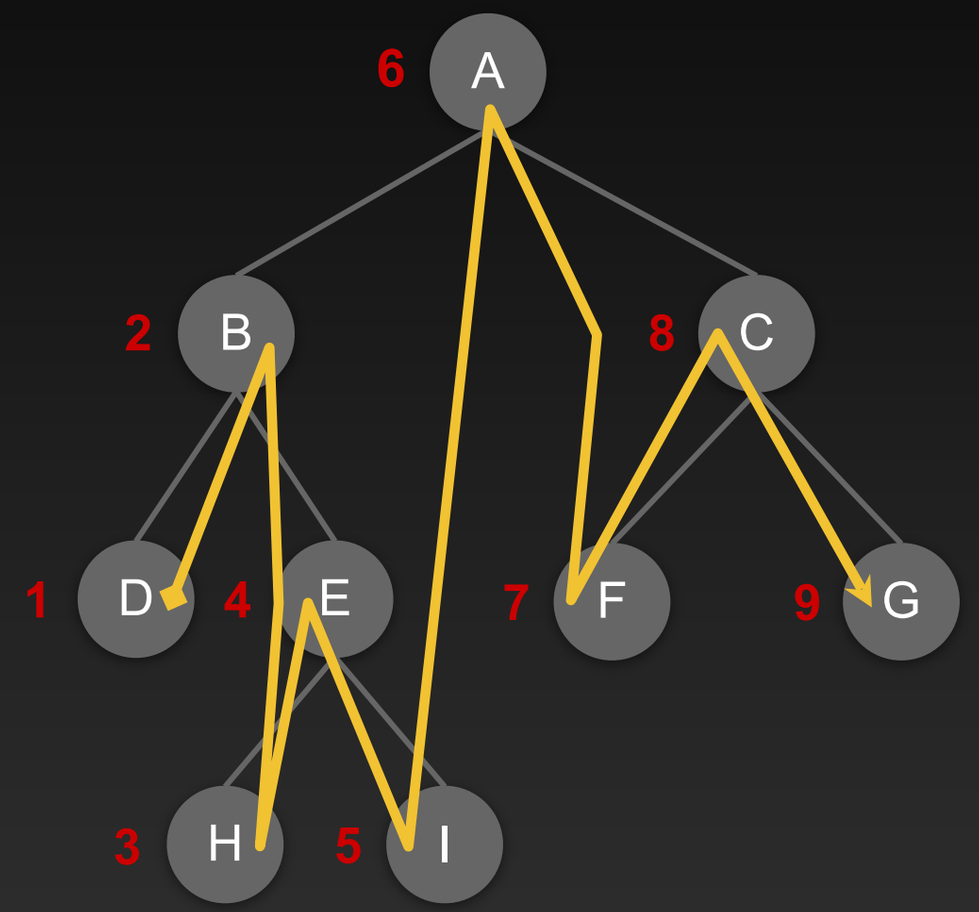
Recorrido en In-orden

Partiendo desde la raíz:

1) Visitar hijo izquierdo en in-orden

2) Visitar nodo actual

3) Visitar hijo derecho en in-orden



El recorrido en este caso sería: D-B-H-E-I-A-F-C-G

Árboles: Recorrido en Profundidad

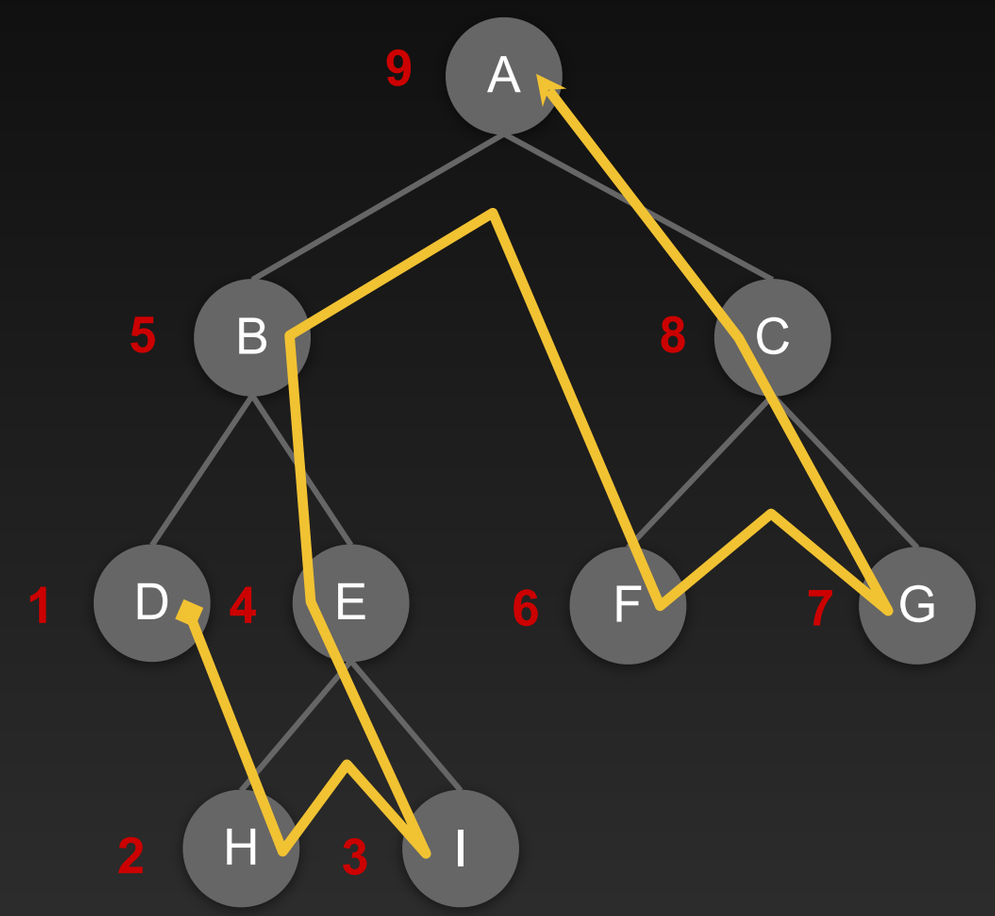
Recorrido en Post-orden

Partiendo desde la raíz:

1) Visitar hijo izquierdo en post-orden

2) Visitar hijo derecho en post-orden

3)Visitar nodo actual



El recorrido en este caso sería: D-H-I-E-B-F-G-C-A

Árboles: Recorrido en Profundidad

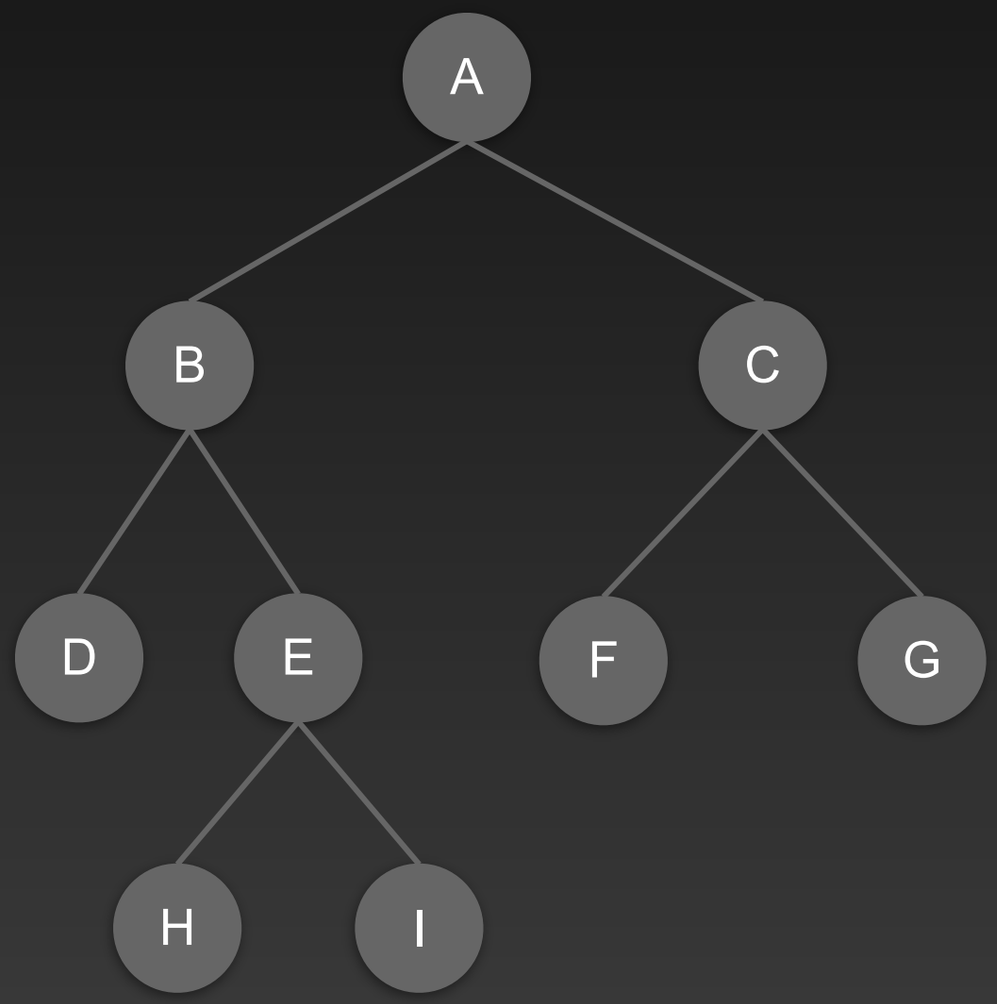
Este tipo de recorridos suelen implementarse de forma recursiva.

Sin embargo, se puede implementar de forma iterativa utilizando pilas como estructuras de datos auxiliares.

Árboles Binarios

Árboles Binarios

- Es un árbol de grado 2.
  - Cada nodo tiene 0, 1 o 2 descendientes directos: el hijo izquierdo y el hijo derecho (que a su vez son árboles binarios)



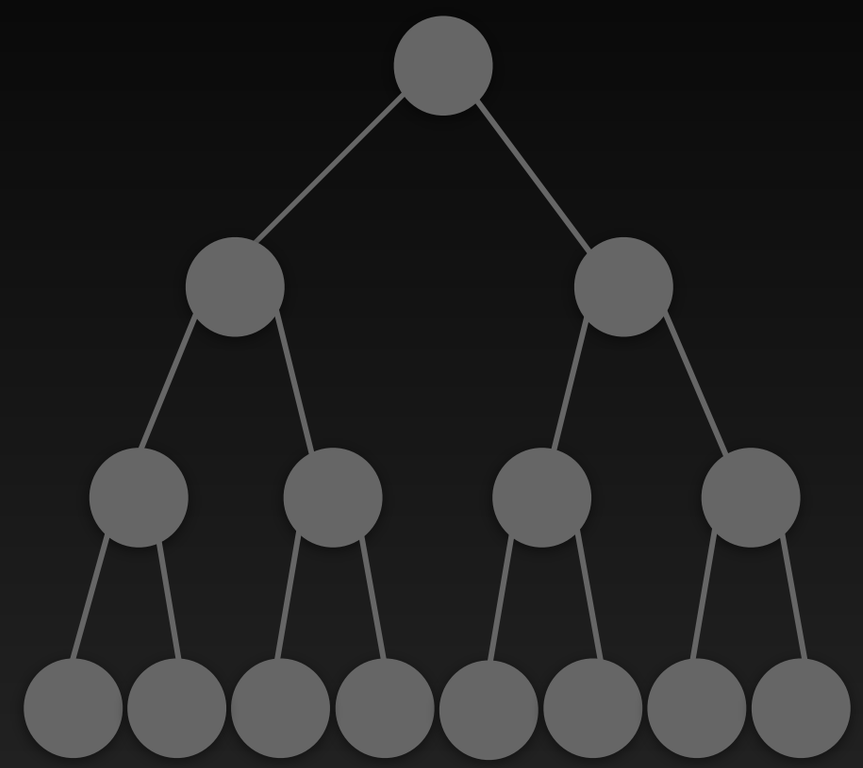
Árboles Binarios: Propiedades

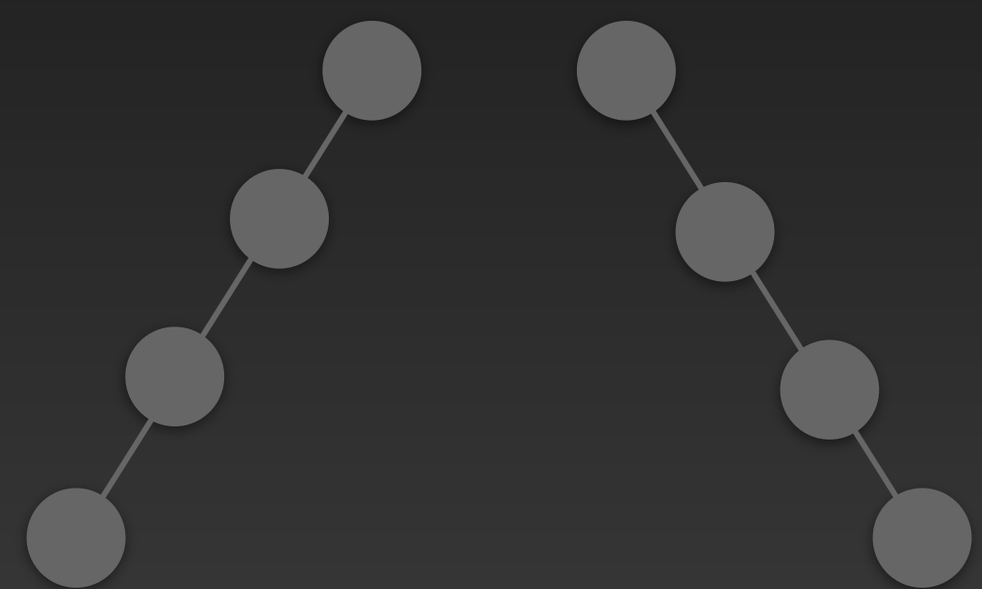
Algunas de las propiedades más importantes de los árboles binarios son las siguientes:

- $e \leq 2^h$
- $\log_2(n+1) - 1 \leq h \leq n - 1$

Donde:

- n: es la cantidad de nodos del árbol,
- h: la altura del árbol, y
- e: la cantidad de nodos externos del árbol.



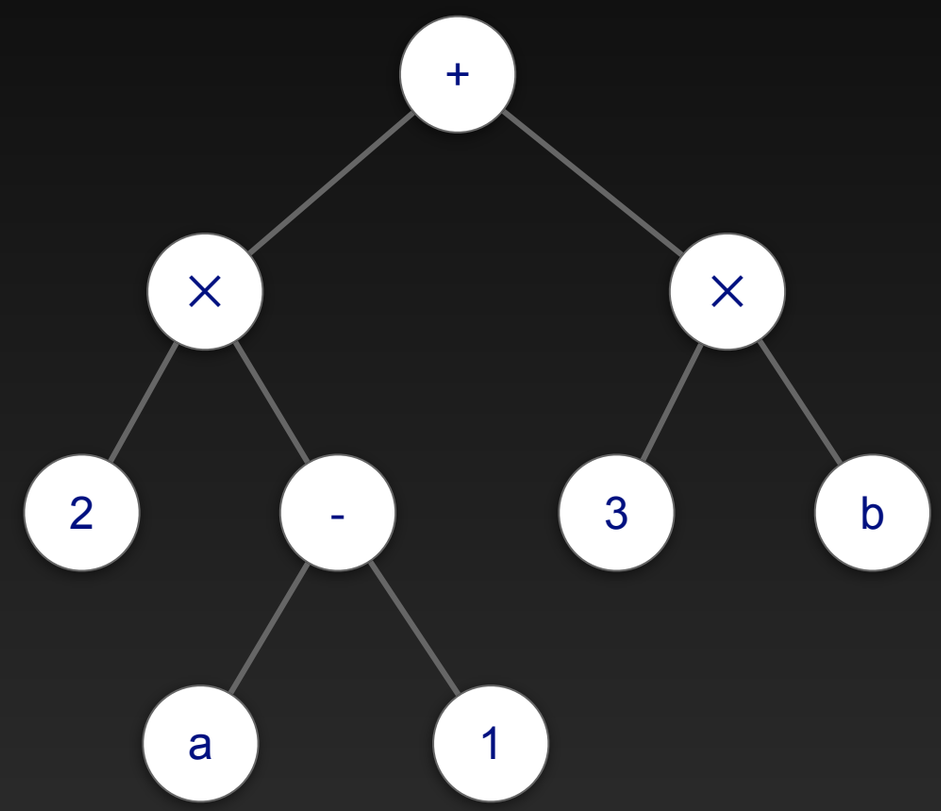


Árboles Binarios: Algunas Aplicaciones

Aplicación 1

Expresiones aritméticas

- **Nodos internos:** operadores.
- **Nodos externos:** operandos.



Por ejemplo,

El árbol nos permitiría representar los paréntesis de la expresión:

$$2 \times (a - 1) + (a \times b)$$

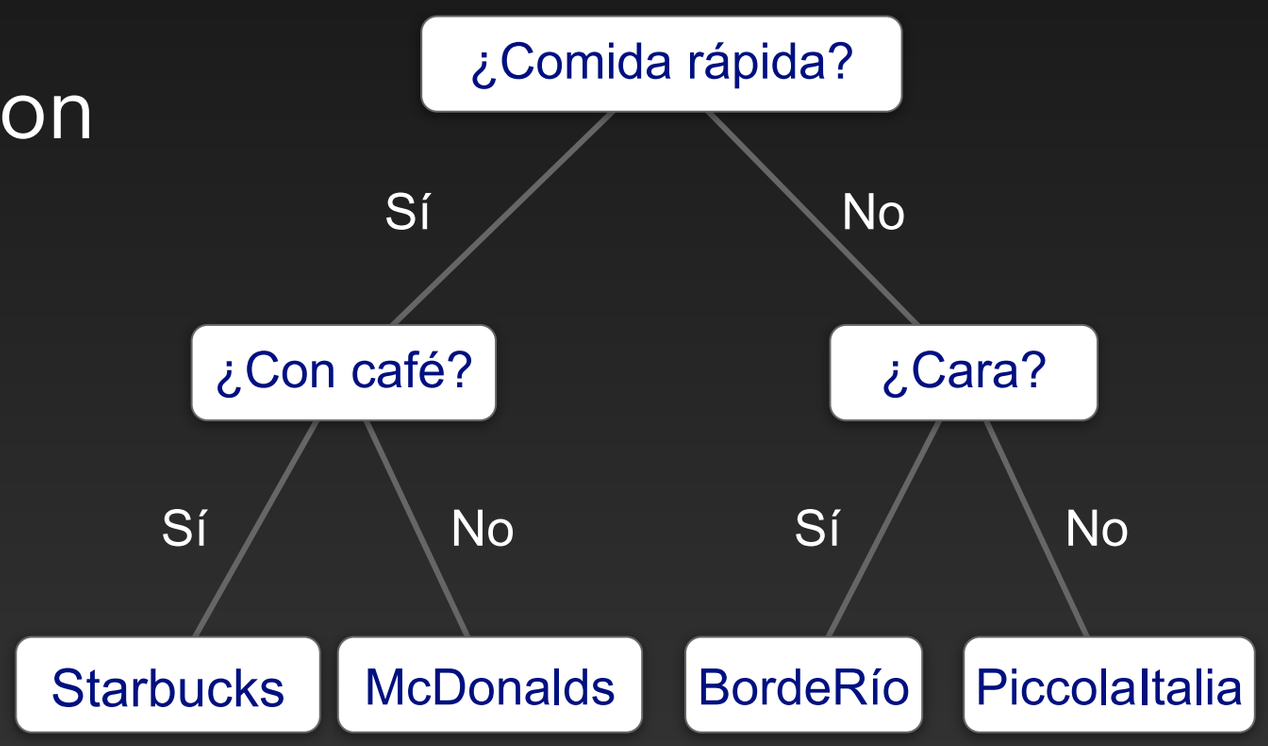
Árboles Binarios: Algunas Aplicaciones

Aplicación 2: Árboles de Decisión

Los árboles de decisión nos permiten modelar posibles alternativas, con preguntas del estilo SI/NO.

- **Nodo interno:** preguntas con respuestas si/no.
- **Nodos hojas:** decisiones.

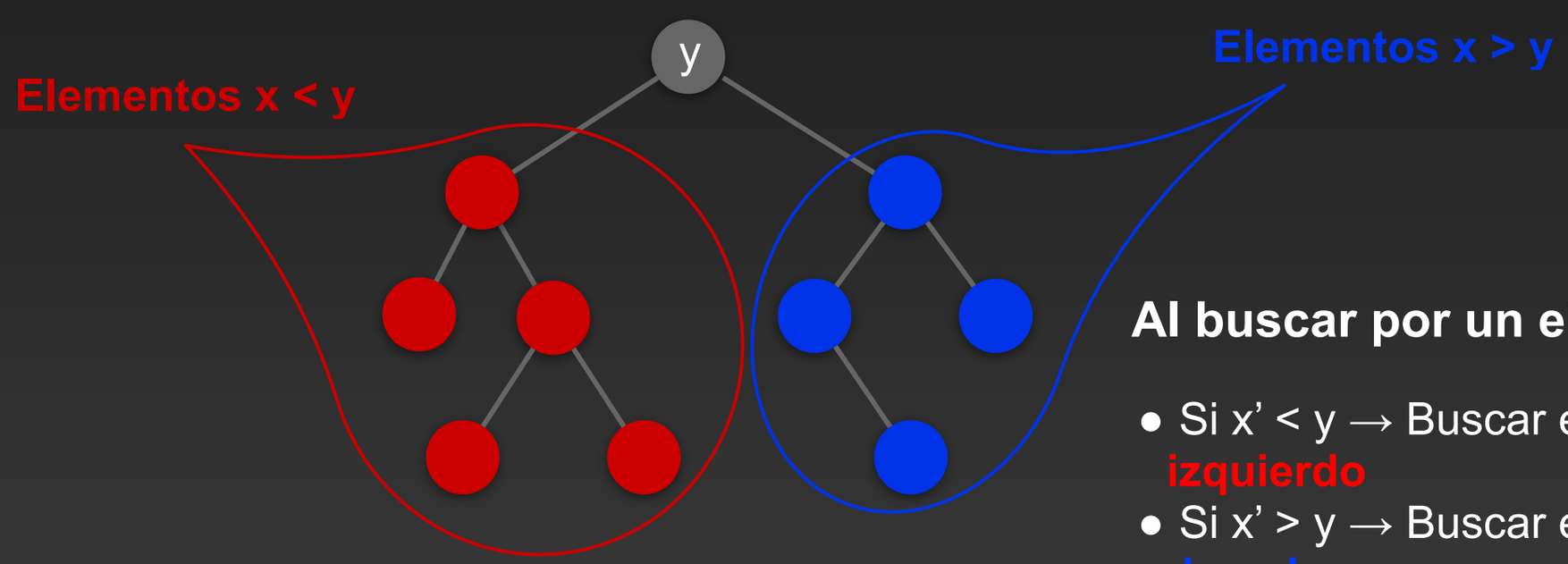
Ejemplo:
¿Dónde comemos?



Árboles Binarios: Aplicaciones

Aplicación 3 Árboles de Búsqueda

- Una de las aplicaciones más importantes de los árboles es la **búsqueda de elementos** dentro de un conjunto de datos.
- Estos tienen como se llaman **árboles binarios de búsqueda (ABB)** o **binary search trees (BST).**
- Estos permiten la búsqueda eficiente, y para permitir aquello, los datos se tienen que organizar de la siguiente forma.

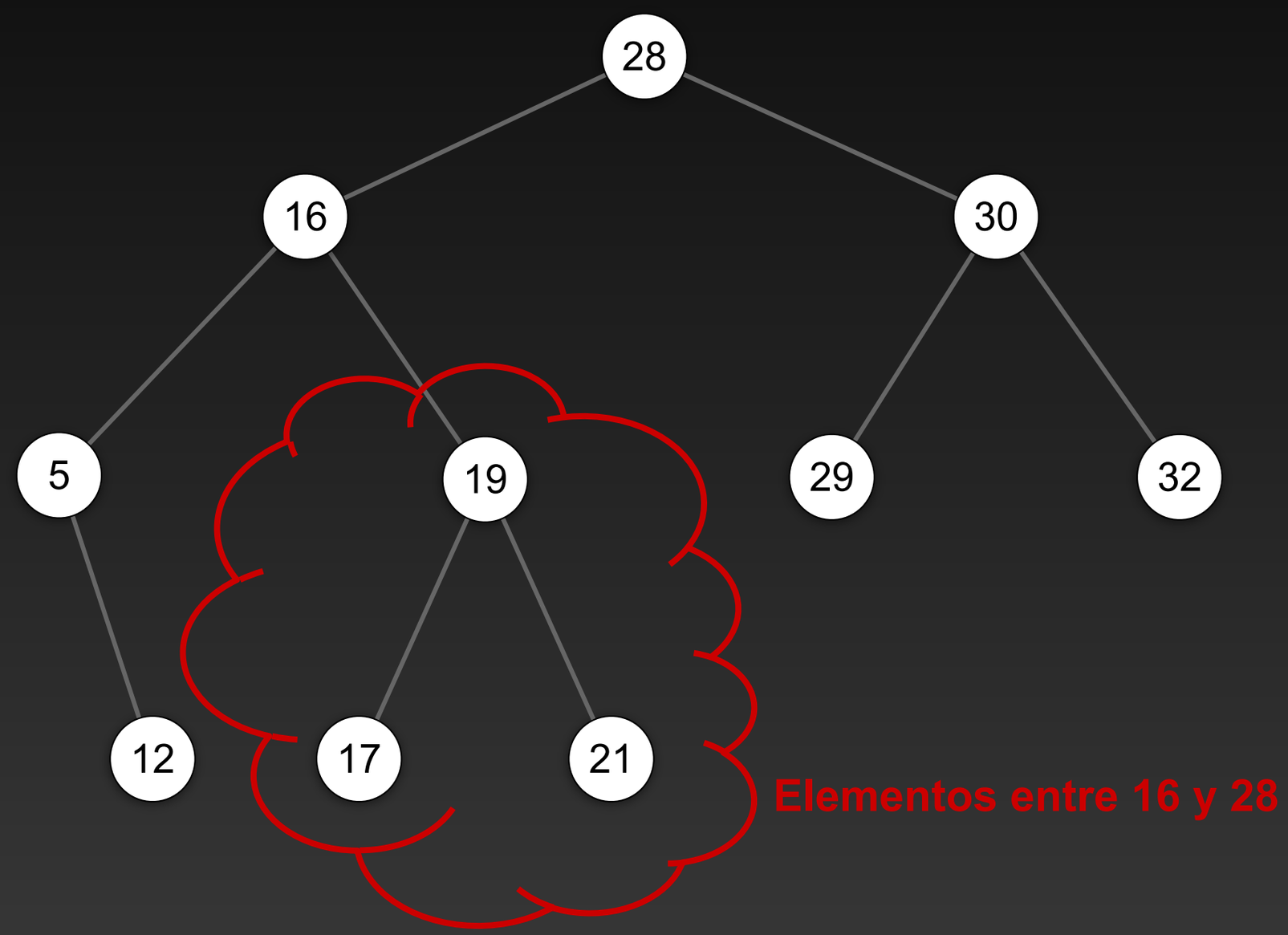


Al buscar por un elemento x':

- Si x' < y → Buscar en **subárbol izquierdo**
- Si x' > y → Buscar en **subárbol derecho**

Árboles Binarios de Búsqueda

Ejemplo:



Binary Search Trees (BST)

Ejemplo: construir el ABB correspondiente a la siguiente secuencia:

10 - 5 - 7 - 14 - 12 - 18 - 15

Binary Search Trees

Para 10 - 5 - 7 - 14 - 12 - 18 - 15, comenzamos con el árbol vacío:

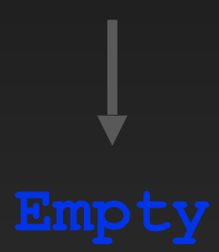
Árbol vacío

`Empty`

Binary Search Trees

Para ~~10~~ - 5 - 7 - 14 - 12 - 18 - 15

Árbol vacío

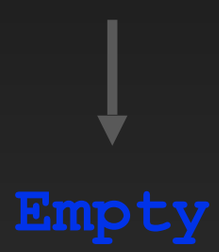


Insertar 10

Binary Search Trees

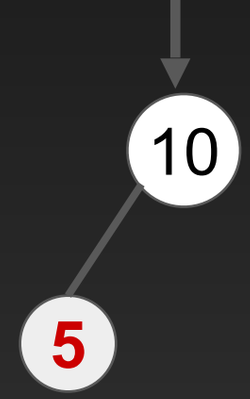
Para ~~10~~ - ~~5~~ - 7 - 14 - 12 - 18 - 15

Árbol vacío



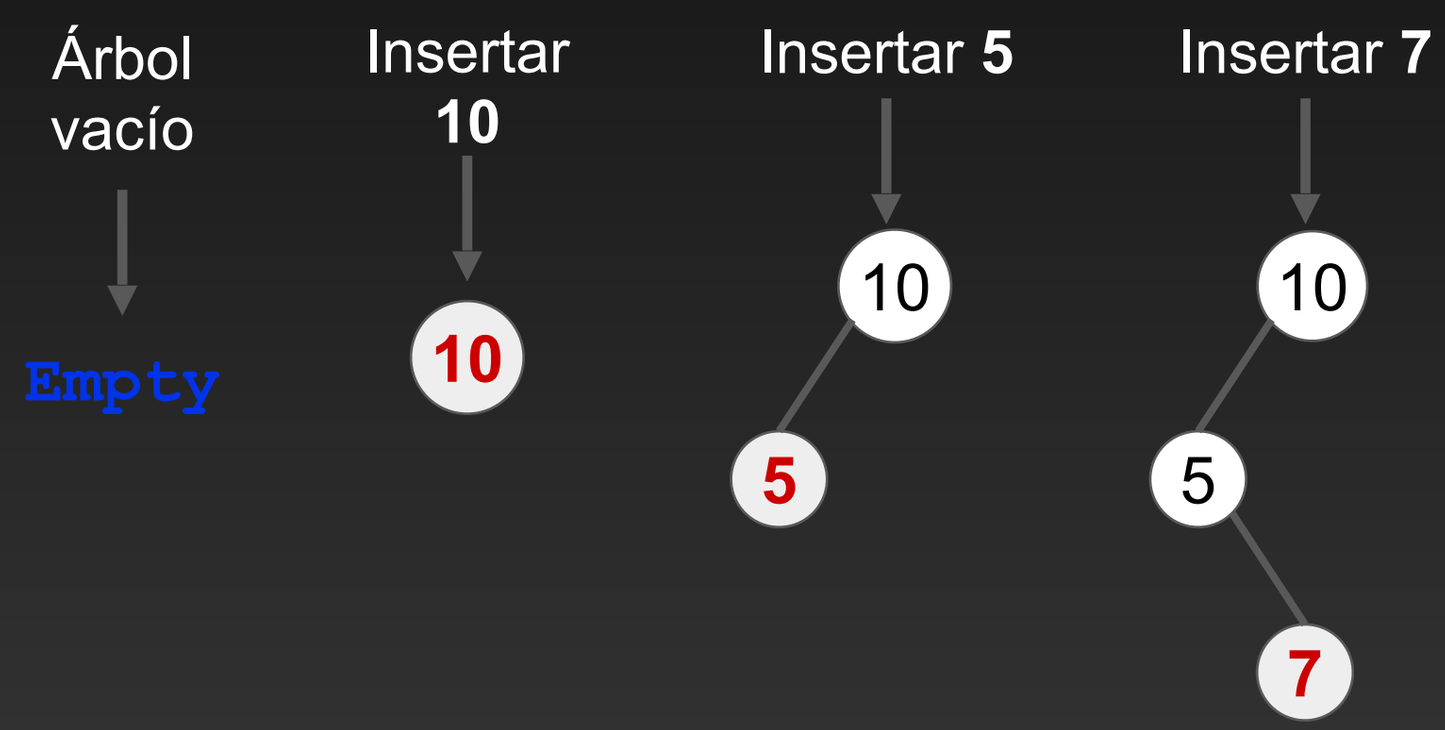
Insertar **10**

Insertar **5**



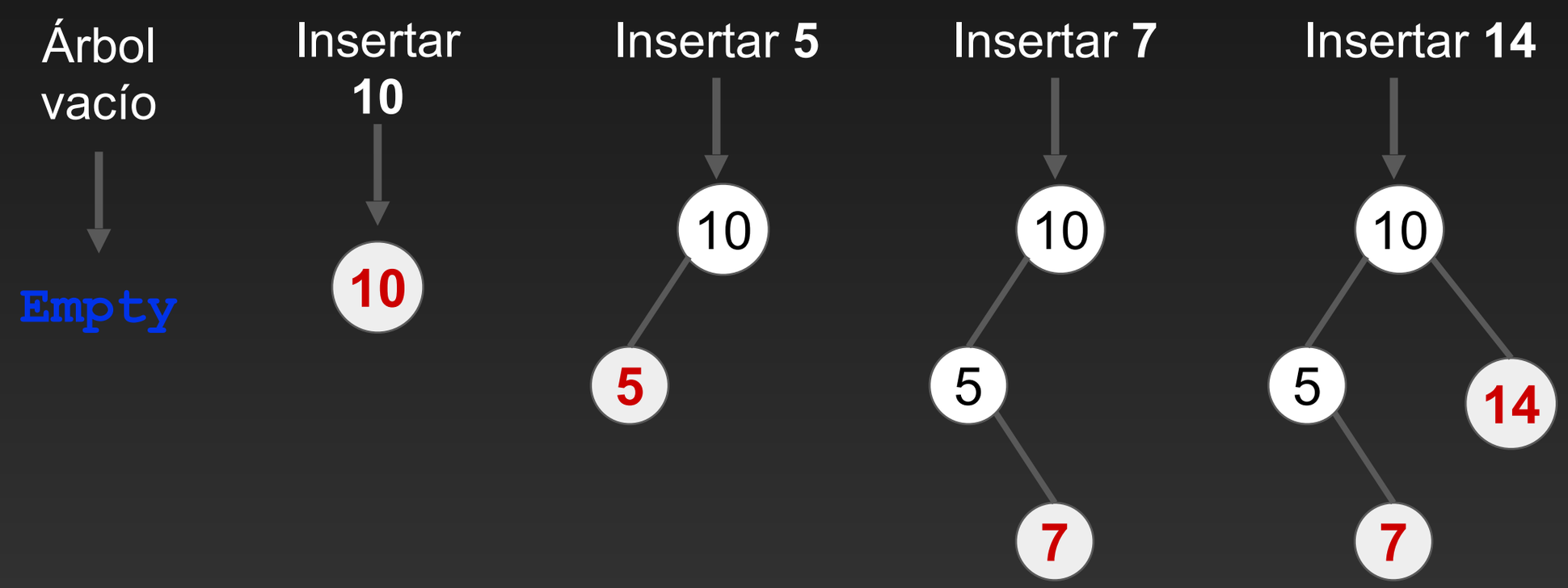
Binary Search Trees

Para ~~10~~ - ~~5~~ - ~~7~~ - 14 - 12 - 18 - 15



Binary Search Trees

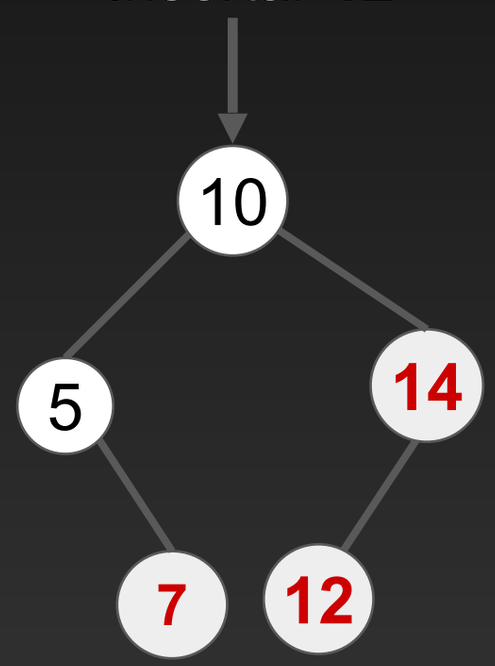
Para ~~10~~ - ~~5~~ - ~~7~~ - ~~14~~ - 12 - 18 - 15



Binary Search Trees

Para ~~10~~ - ~~5~~ - ~~7~~ - ~~14~~ - ~~12~~ - 18 - 15

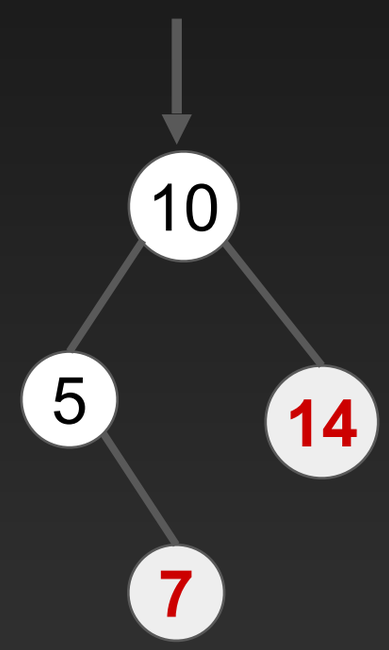
Insertar **12**



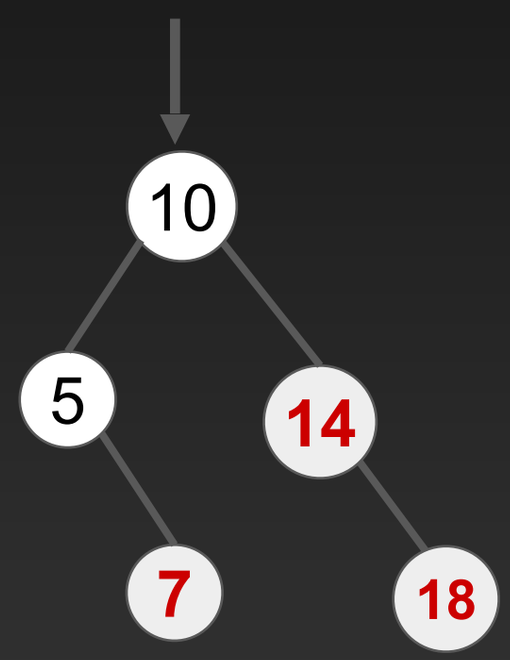
Binary Search Trees

Para ~~10~~ - ~~5~~ - ~~7~~ - ~~14~~ - ~~12~~ - ~~18~~ - 15

Insertar **12**



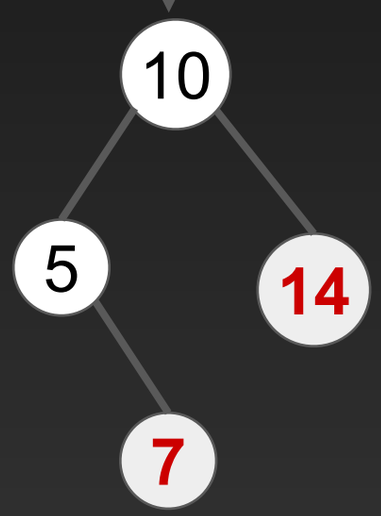
Insertar **18**



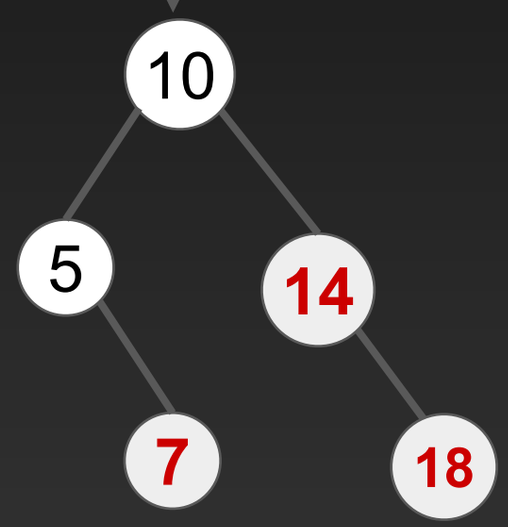
Binary Search Trees

Para ~~10~~ - ~~5~~ - ~~7~~ - ~~14~~ - ~~12~~ - ~~18~~ - ~~15~~

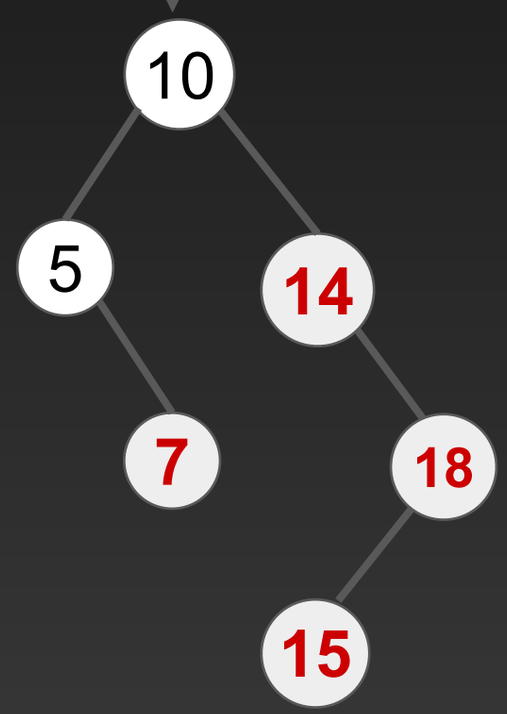
Insertar **12**



Insertar **18**



Insertar **15**



Binary Search Trees

Costo de las Operaciones

La complejidad de búsqueda de un elemento en un ABB depende de la profundidad en la que se encuentre el nodo correspondiente

- Un nodo muy profundo es más costoso de encontrar
- Pero, ¿cuán profundo puede encontrarse un nodo en un ABB?
- Para acotar las complejidades asintóticamente, conviene recordar las propiedades de altura de los árboles binarios
- Altura mínima de un árbol de n nodos es $O(\log_2 n)$
- Altura máxima de un árbol de n nodos es $O(n)$

Binary Search Trees

Costo de las Operaciones

La complejidad de búsqueda de un elemento en un ABB depende de la profundidad en la que se encuentre el nodo correspondiente

- Un nodo muy profundo es más costoso de encontrar
- Pero, ¿cuán profundo puede encontrarse un nodo en un ABB?
- Para acotar las complejidades asintóticamente, conviene recordar las propiedades de altura de los árboles binarios
- Altura mínima de un árbol de n nodos es $O(\log_2 n)$
- Altura máxima de un árbol de n nodos es $O(n)$

Binary Search Trees

Costo de las Operaciones

- Entonces en peor caso la búsqueda tomaría tiempo $O(n)$
- Esto debido a que la forma del árbol depende del orden en que se inserten los elementos del árbol.
- Para mitigar esto, se han desarrollado lo que se llaman árboles balanceados, lo que permite garantizar que siempre la altura del árbol será logaritimica
  - i.e., $h = O(\log_2(n))$
  - Lo que permite lograr tiempo de búsqueda de $O(\log_2(n))$
- Existen multiples variantes, pero los más famosos son:
  - Árboles Rojo-Negro
  - Árboles AVL
  - Árboles 2-3, etc